Project report

01 Project summary

The Harbor Library Search Pilot is a fictional six-week software project for a small public-library catalog. The team built a search interface, a catalog API and an import process for a synthetic collection of 18,000 catalog records. All organizations, schedules and measurements in this report are invented for testing DocuChat; they are not achievements of the DocuChat project.

The pilot addressed inconsistent search results and slow catalog browsing. It did not replace library circulation software. The final deliverables were a browser interface, a documented API, a repeatable import command and a test report. The pilot ran from 3 August to 11 September 2026. A demonstration used a synthetic dataset rather than real borrower information.

Project duration	Six weeks
Pilot dates	3 August to 11 September 2026
Catalog size	18,000 synthetic records
Excluded system	Library circulation software

Reading note: answer from this document only. If a requested detail is absent, say that it is not specified.

Project report

02 Scope and stakeholders

The pilot supported title, author and subject searches. Readers could filter by publication year and material type. A catalog administrator could replace the synthetic catalog through a validated import. Borrower accounts, loans, fines and reservation handling were outside scope.

The fictional team included a coordinator, two developers and one tester. A library representative reviewed search terminology and result presentation. The coordinator maintained acceptance criteria and scheduled weekly demonstrations. The representative approved the scope but did not administer the test infrastructure. Change requests were logged separately so that an attractive extra feature could not silently replace a required search behavior.

Search fields	Title, author and subject
Filters	Publication year and material type
Team	Coordinator, two developers and one tester
Excluded	Borrower accounts, loans, fines and reservations

Reading note: answer from this document only. If a requested detail is absent, say that it is not specified.

Project report

03 Requirements and acceptance

The catalog search endpoint accepted a query string and optional year and material-type filters. Empty queries returned a browsable catalog page. Each result included a record identifier, title, author, publication year and material type. Results used deterministic ordering to make pagination repeatable.

Acceptance required correct filters, no missing mandatory result fields and a repeatable import. The team prepared forty functional search cases with expected identifiers. A reviewer checked accessibility with keyboard navigation and visible focus states. Response-time targets were recorded before optimization to prevent the team from redefining success after seeing measurements. No requirement included machine-learning recommendations or personalized ranking.

Functional search cases	Forty
Mandatory result fields	Identifier, title, author, year and type
Ordering	Deterministic
Not included	Personalized ranking

Reading note: answer from this document only. If a requested detail is absent, say that it is not specified.

Project report

04 System architecture

The fictional pilot used a static browser interface, an HTTP catalog API and a relational database. The browser sent queries to the API rather than connecting directly to the database. Database credentials remained on the server. The import command validated a CSV file before inserting catalog records.

The service separated request validation, search queries and response formatting into different modules. A database index supported common search filters. The system used structured logs to record request duration and status. The report describes the Harbor pilot architecture only; it must not be confused with DocuChat, which uses a Gemini-backed retrieval pipeline and optional Qdrant storage.

Components	Browser, HTTP API and relational database
Import format	CSV
Secret boundary	Database credentials on server
Logging	Request duration and response status

Reading note: answer from this document only. If a requested detail is absent, say that it is not specified.

Project report

05 Schedule and milestones

Week one covered requirements and a synthetic dataset. Week two delivered catalog import validation and the initial database schema. Week three delivered search endpoints and pagination. Week four connected the browser interface and filters. Week five focused on optimization and accessibility. Week six covered regression checks, deployment rehearsal and handover.

Each milestone included a brief demonstration and a written issue list. An incomplete feature carried into the next week with an explicit owner. The team postponed a proposed recommendation panel because it was outside the accepted scope. The schedule left the last week for verification rather than treating the first successful demonstration as the end of the project.

Week two	Import validation and schema
Week three	Search endpoints and pagination
Week five	Optimization and accessibility
Week six	Regression, rehearsal and handover

Reading note: answer from this document only. If a requested detail is absent, say that it is not specified.

Project report

06 Testing approach

The final test suite contained 68 automated cases: forty search cases, sixteen import-validation cases and twelve API error-handling cases. The suite used a disposable database loaded from the synthetic catalog fixture. Tests did not depend on real borrower records or a production account.

The tester separately checked keyboard navigation, empty states and readable error messages. Automated cases verified expected record identifiers rather than only checking for HTTP 200. Import tests covered missing required columns, duplicate identifiers and invalid years. A successful local suite established behavior in that environment; the team still ran a deployment smoke check and restoration rehearsal afterward.

Automated cases	68 total
Search cases	40
Import cases	16
API error cases	12

Reading note: answer from this document only. If a requested detail is absent, say that it is not specified.

Project report

07 Search performance results

Under the same synthetic workload, the measured p95 search API response time fell from 780 milliseconds before optimization to 310 milliseconds after optimization. The workload used twenty concurrent simulated users against 18,000 catalog records. Measurements excluded browser rendering and covered the API request duration only.

The optimization added an index for frequent filters, removed repeated per-result database queries and limited the returned fields. The test environment used two virtual CPU cores and 4 GB of memory. The report records these results as fictional sample data. They must not be quoted as DocuChat performance. Average latency was not recorded, so it cannot be derived from the p95 values.

Before optimization	p95 780 ms
After optimization	p95 310 ms
Concurrency	Twenty simulated users
Environment	Two virtual CPU cores and 4 GB memory

Reading note: answer from this document only. If a requested detail is absent, say that it is not specified.

Project report

08 Issues and corrective actions

An early import accepted duplicate catalog identifiers and caused inconsistent result counts. The team added a uniqueness constraint and a validation error listing duplicate identifiers before commit. This made invalid imports fail as a whole rather than publishing a partially replaced catalog.

A second issue reset filters when moving between result pages. The client began carrying filter parameters into every pagination request. A regression case verified that a year filter remained active on the second page. The team also corrected a keyboard focus trap in the filter panel. Each issue was linked to a reproduction procedure and a verification check, not closed solely because a developer had changed code.

Import issue	Duplicate catalog identifiers
Import fix	Uniqueness constraint and pre-commit validation
Pagination issue	Filters reset between pages
Accessibility issue	Keyboard focus trap

Reading note: answer from this document only. If a requested detail is absent, say that it is not specified.

Project report

09 Deployment and handover

The team rehearsed deployment in a staging environment using a fresh database and the documented import command. Configuration values were supplied through server environment variables. The handover package contained setup steps, an environment-variable checklist, API examples and a recovery procedure.

A reviewer restarted the staging API and checked that catalog records remained available. The reviewer also restored the synthetic catalog into a separate database from an export. These checks demonstrated behavior in staging only; they did not prove long-term production reliability. The report includes no public service URL or service-level agreement. Ownership after handover passed to the fictional catalog maintenance team.

Environment	Staging
Persistence check	Restart API and query catalog
Recovery check	Restore export to a separate database
Handover owner	Catalog maintenance team

Reading note: answer from this document only. If a requested detail is absent, say that it is not specified.

Project report

10 Conclusions and next steps

The pilot met its recorded functional acceptance cases and improved search latency in the defined synthetic environment. The main engineering lesson was to connect claims to repeatable evidence: expected result identifiers, fixed test data, workload details and a clear separation between local, staging and production verification.

Proposed next steps were typo-tolerant search, richer catalog descriptions and larger workload tests. These proposals were not implemented during the six-week pilot. Personalized recommendations remained outside scope because they would need an evaluation dataset and additional design work. The report does not provide user-satisfaction scores, deployment costs or production traffic figures. A question about those missing measurements should receive an explicit statement that the report does not contain them.

Completed	Recorded functional checks and staging rehearsal
Proposed	Typo tolerance and larger workload tests
Not measured	User satisfaction and production traffic
Not provided	Deployment costs

Reading note: answer from this document only. If a requested detail is absent, say that it is not specified.